Taller 4: gestores de estado - persistencia local

Autor

Daniela Erazo Marin

Docente

David Steven Duran Vallejo

Unidad Central Del Valle

Facultad De Ingeniería

Ingeniería De Sistemas

Tuluá – Valle Del Cauca

2025

Repo: https://github.com/daniela-erazo-marin/curso_flutter

1️⃣ Objetivo

Evaluar las habilidades del desarrollador en el uso de **Flutter** para construir una aplicación móvil moderna con:

- Gestión de estado y arquitectura limpia.
- Integración con **API REST**.
- **Persistencia local** usando SQLite (sqflite) y modo offline con sincronización.
- Buenas prácticas de desarrollo (documentación, control de versiones).

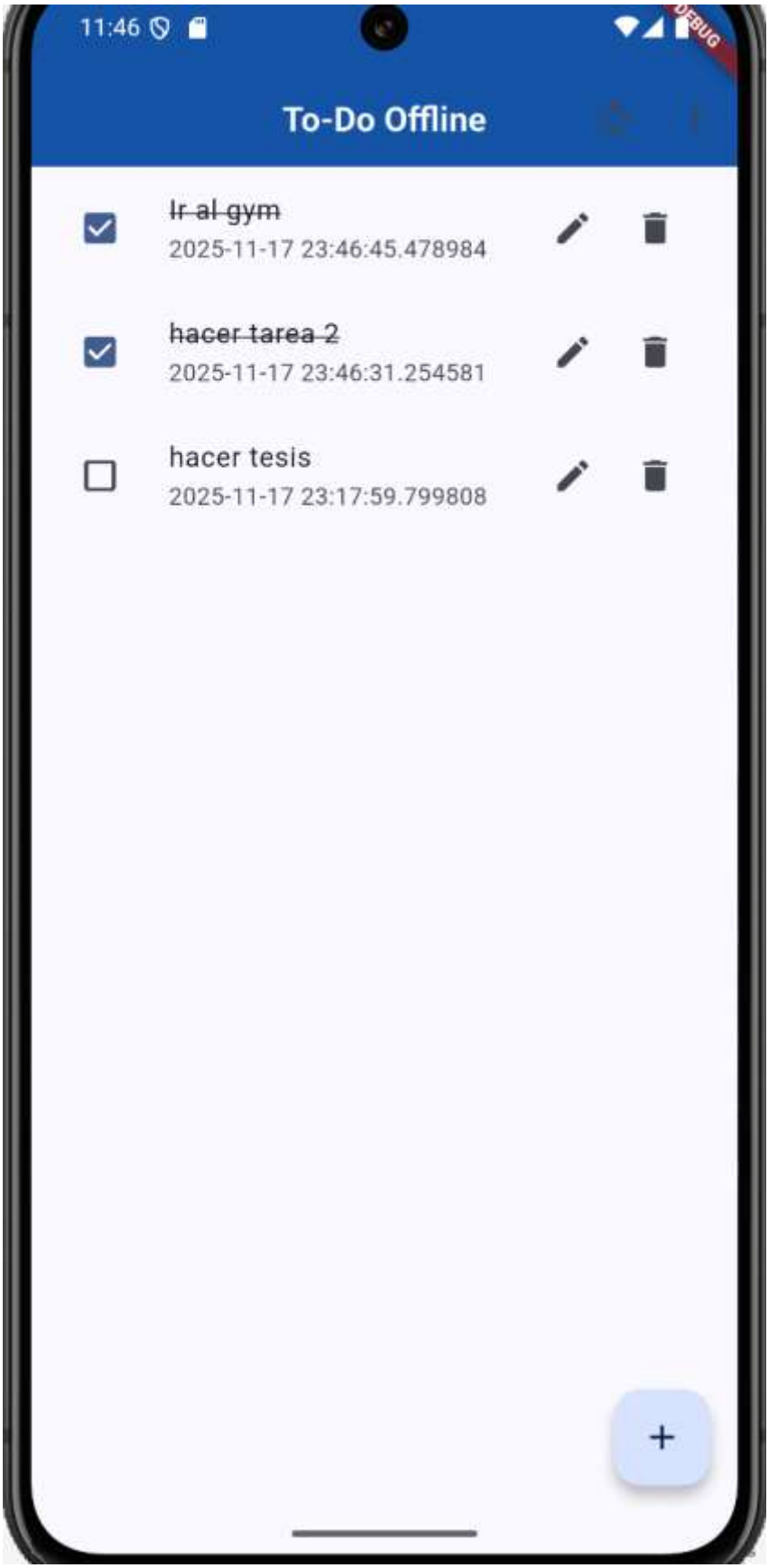
2️⃣ Aplicación: To-Do List

Desarrollar una aplicación Flutter que permita:

- Crear, editar, marcar como completadas y eliminar tareas.
- Listar tareas con filtros básicos (todas, pendientes, completadas).


11:45
DEBUG
To-Do Offline
hacer tarea 2
2025-11-17 23:19:58.901656
hacer tesis
2025-11-17 23:17:59.799808


11:46
DEBUG
Editar tarea
Título
hacer tarea 2
Actualizar


11:46
DEBUG
To-Do Offline
Ir al gym
2025-11-17 23:46:45.478984
hacer tarea 2
2025-11-17 23:46:31.254581
hacer tesis
2025-11-17 23:17:59.799808

3 Integración con API

Se crea una FastAPI con Python cumpliendo con el contrato de endpoints.

Endpoints esperados

GET /tasks

POST /tasks

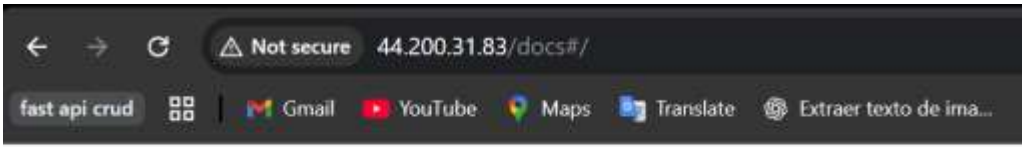
GET /tasks/{id}

PUT /tasks/{id}

DELETE /tasks/{id}

Formato: JSON

Campos mínimos: id, title, completed (bool), updatedAt (ISO8601).

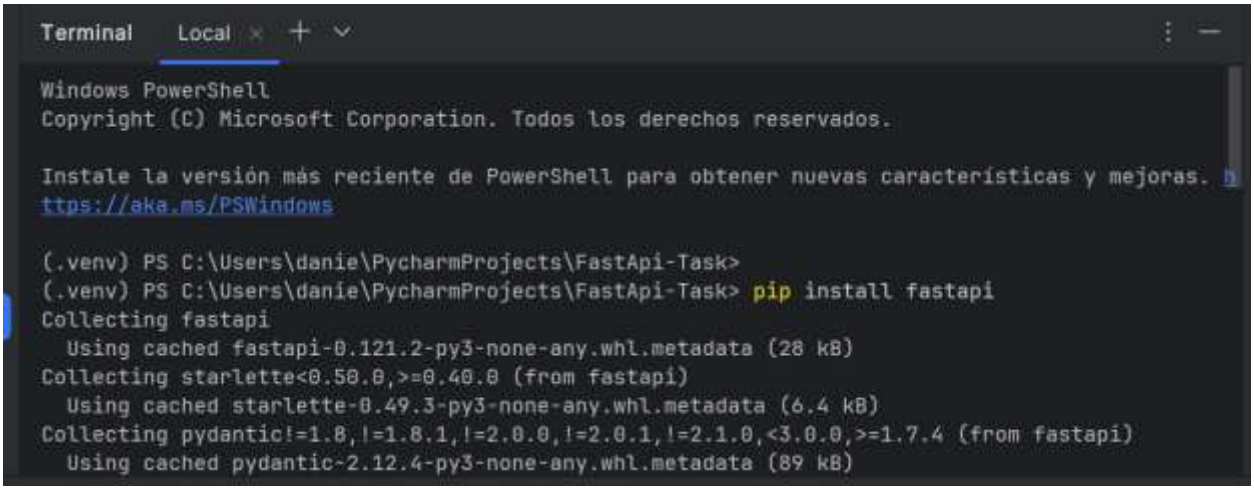


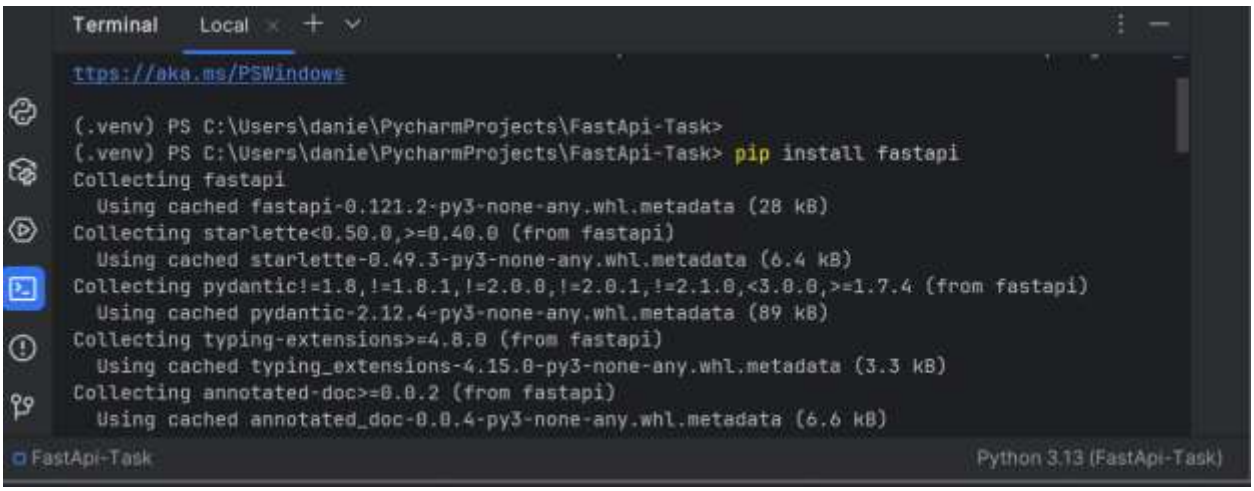
Código de la Fast API:

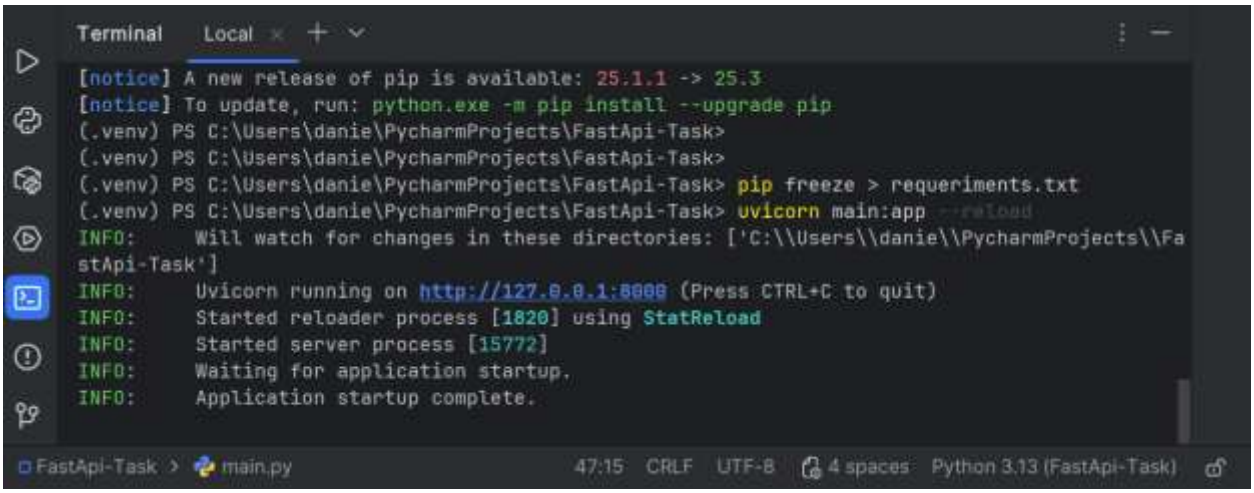
requirements.txt main.py

```python

@app.post("/tasks")  Daniela E
def save_task(task: Task):
    # Evitar IDs duplicados
    for existing in tasks:
        if existing["id"] == task.id:
            raise HTTPException(status_code=400, detail="ID already exists")

    task.updated_at = datetime.now()
    tasks.append(task.dict())
    return task


@app.get("/tasks/{task_id}")  Daniela E
def get_task(task_id: str):
    for task in tasks:
        if task["id"] == task_id:
            return task
    raise HTTPException(status_code=404, detail="Item not found")


@app.delete("/tasks/{task_id}")  Daniela E
def delete_task(task_id: str):
    for index, task in enumerate(tasks):
        if task["id"] == task_id:
            tasks.pop(index)
            return {"message": "Task deleted successfully"}
    raise HTTPException(status_code=404, detail="Item not found")


@app.put("/tasks/{task_id}")  Daniela E
def update_task(task_id: str, updatedTask: Task):
```

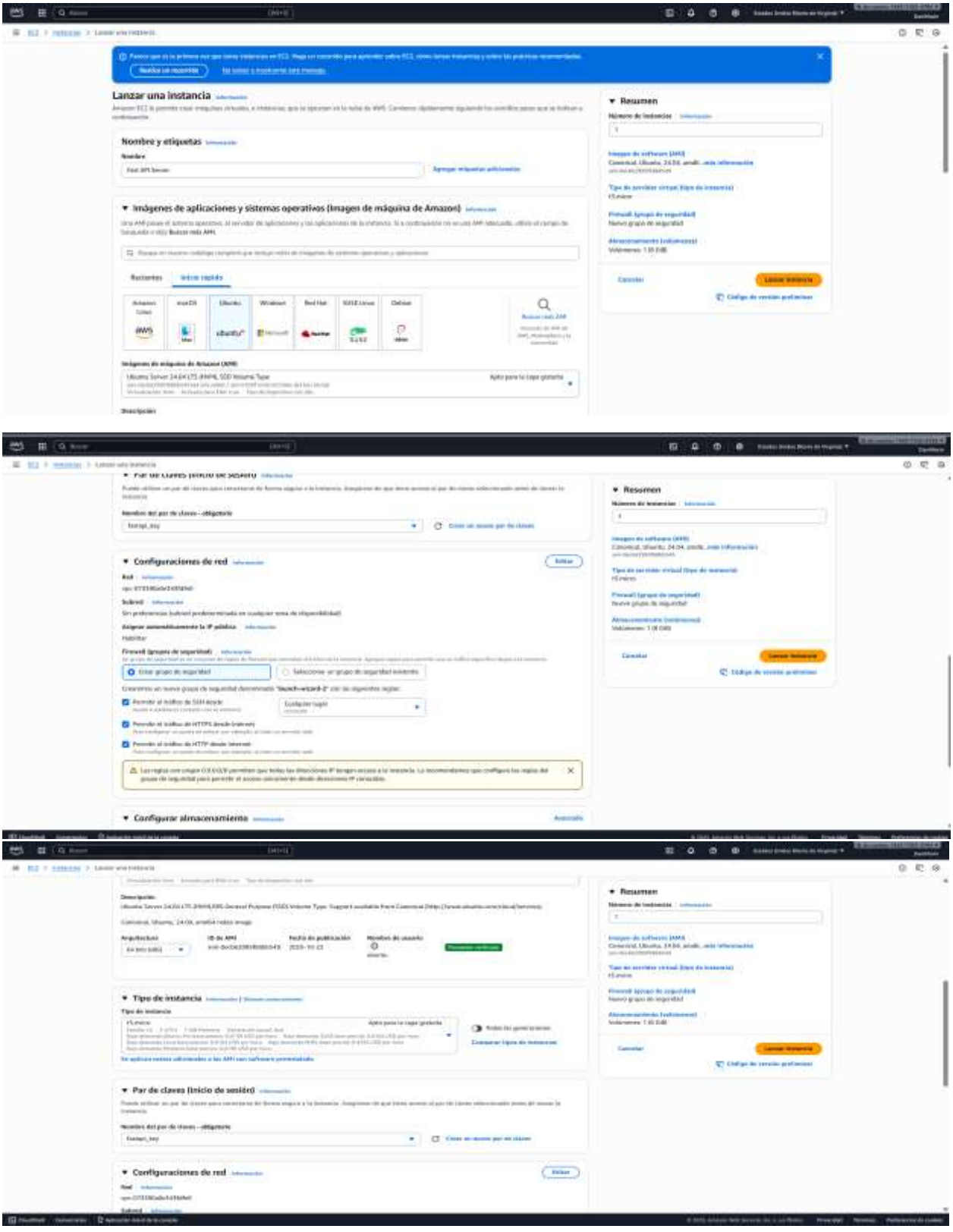
Comandos usados para crear la FastApi-Task

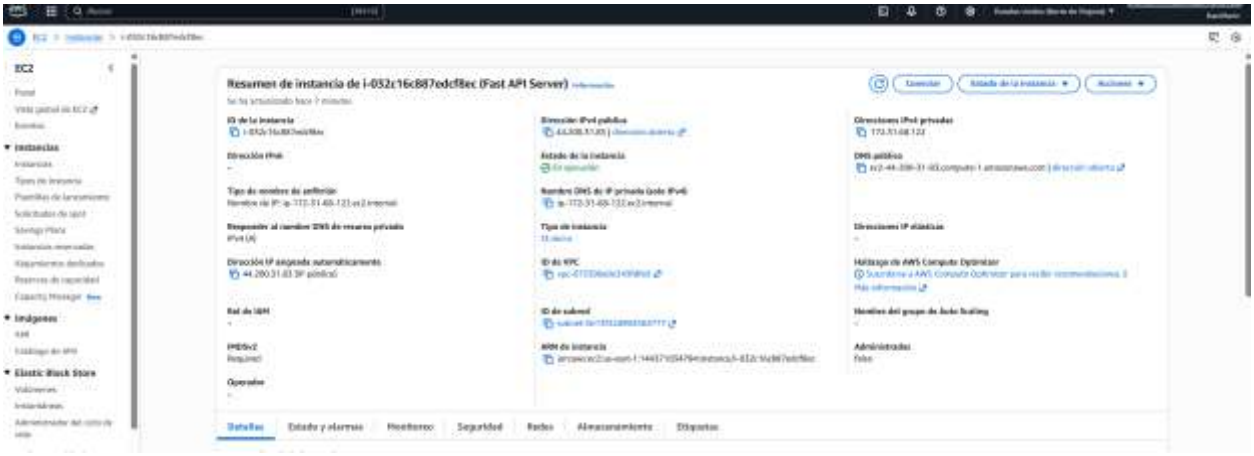




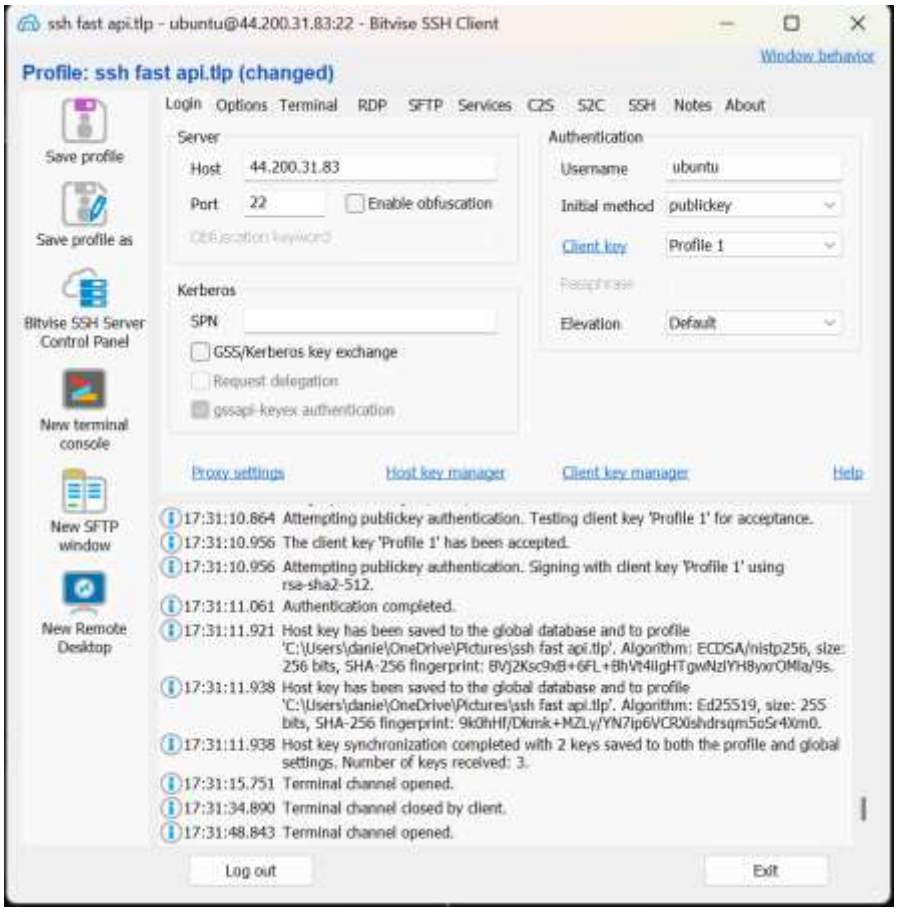


Configuración en AWS





Me conecte a la maquina Ubuntu por medio de ssh



Comandos usados:

```
Last login: Mon Nov 17 00:47:48 2025 from 190.108.78.184
ubuntu@ip-172-31-68-122:~$ history
    1  sudo apt-get update
    2  cls
    3  clc
    4  clear
    5  sudo apt install -y python3-pip nginx
    6  sudo vim /etc/nginx/sites-enable
    7  sudo vim /etc/nginx/sites-enable/fastapi_nginx
    8  history
    9  sudo apt update
   10  cd ~/FastApi-Task
   11  python3 -m venv venv
   12  sudo apt install -y python3.12-venv
   13  rm -rf venv
   14  python3 -m venv venv
   15  source venv/bin/activate
   16  ls
   17  cat requirements.txt
   18  pip install -r requirements.txt
   19  clear
   20  python3 -m uvicorn main:app
   21  cd FastApi-Task/
   22  python3 -m uvicorn main:app
   23  source venv/bin/activate
   24  python3 -m uvicorn main:app
   25  pwd
   26  ls
   27  ls -la
   28  ls -la venv/bin
   29  realpath .
   30  sudo nano /etc/systemd/system/fastapi.service
   31  sudo systemctl daemon-reload
   32  sudo systemctl enable fastapi
   33  sudo systemctl start fastapi
   34  sudo systemctl status fastapi
   35  history
   36  c..
   37  c ..
   38  history
ubuntu@ip-172-31-68-122:~$
```

Fastapi.service corriendo en la instancia ec2 con daemon

bash Copy code

```
sudo nano /etc/systemd/system/fastapi.service
```

ini Copy code

```
[Unit]
Description=FastAPI Service
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/FastApi-Task
ExecStart=/home/ubuntu/FastApi-Task/venv/bin/uvicorn main:app --host 0.0.0.0 --port 8000
Restart=always
RestartSec=3
Environment="PATH=/home/ubuntu/FastApi-Task/venv/bin"

[Install]
WantedBy=multi-user.target
```

bash Copy code

```
sudo systemctl daemon-reload
sudo systemctl enable fastapi
sudo systemctl start fastapi
```

bash Copy code

```
sudo systemctl status fastapi
```

Chat gpt usado para errores:

https://chatgpt.com/share/691a7558-e3e4-8001-b7a0-e51e1d88013e

Video guía para crear instancia en ec2:

https://www.youtube.com/watch?v=SgSnz7kW-Ko

Repo guía:

https://github.com/pixegami/fastapi-tutorial

🗄️ Persistencia local (SQLite)

Usar el paquete sqflite para almacenar las tareas localmente.

Esquema de ejemplo

```
CREATE TABLE tasks (
 id TEXT PRIMARY KEY,
 title TEXT NOT NULL,
 completed INTEGER NOT NULL DEFAULT 0,
 updated_at TEXT NOT NULL,
 deleted INTEGER NOT NULL DEFAULT 0
);
```

```dart
class TaskDB {
  TaskDB._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('tasks.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);
    return await openDatabase(path, version: 1, onCreate: _createDB);
  }

  Future _createDB(Database db, int version) async {
    await db.execute('''
      CREATE TABLE tasks (
        id TEXT PRIMARY KEY,
        title TEXT NOT NULL,
        completed INTEGER NOT NULL DEFAULT 0,
        updated_at TEXT NOT NULL,
        deleted INTEGER NOT NULL DEFAULT 0
      );
    ''');
```

5️⃣ Sincronización (cuando vuelve la conexión)

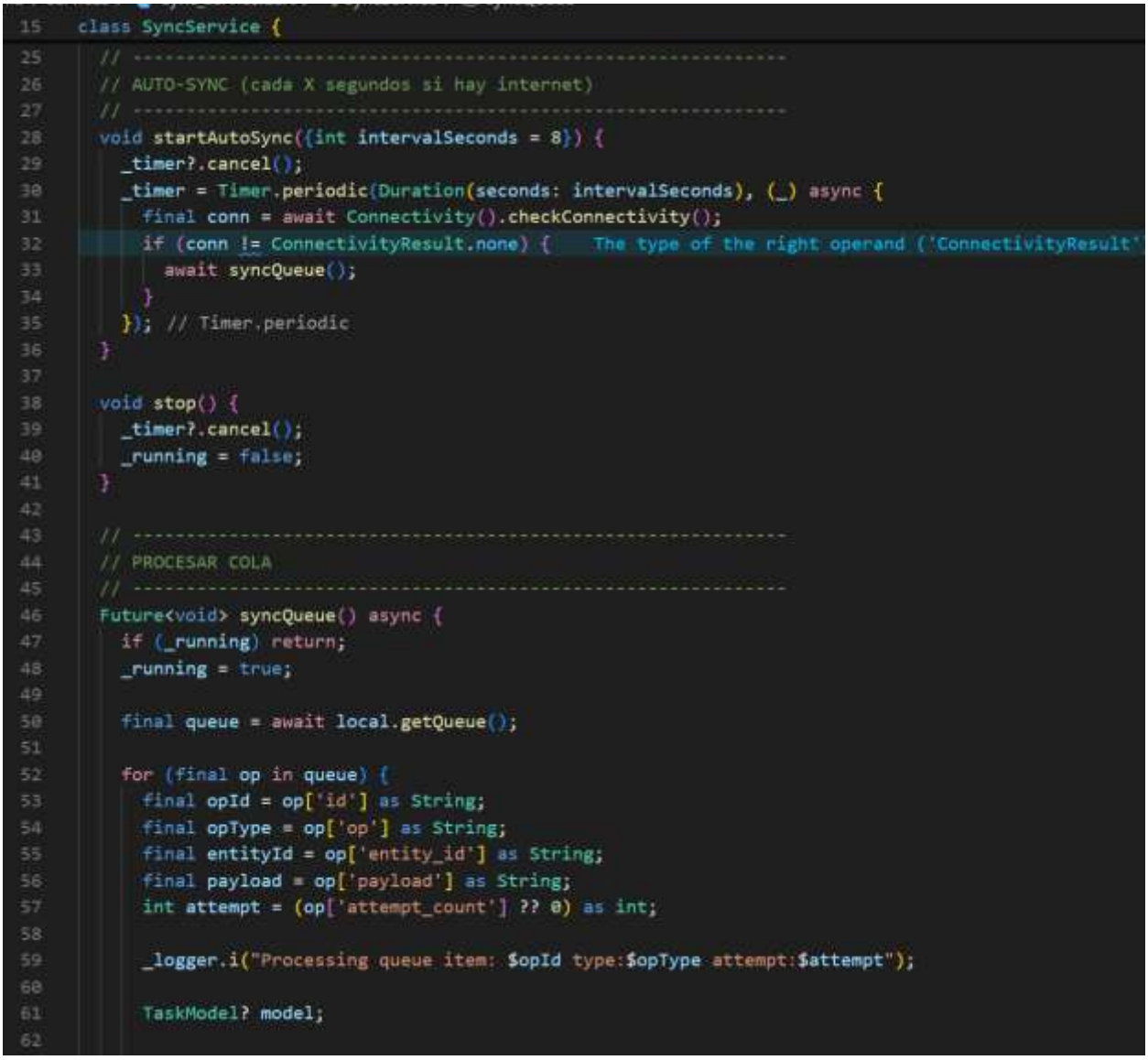
Implementar una tabla de cola de operaciones para sincronizar con el servidor:

```
queue_operations(
 id TEXT PRIMARY KEY,
 entity TEXT,
 entity_id TEXT,
 op TEXT,          -- CREATE | UPDATE | DELETE
 payload TEXT,
 created_at INTEGER,
 attempt_count INTEGER,
 last_error TEXT
);
```

```
    await db.execute('''
      CREATE TABLE queue_operations (
        id TEXT PRIMARY KEY,
        entity TEXT,
        entity_id TEXT,
        op TEXT,
        payload TEXT,
        created_at INTEGER,
        attempt_count INTEGER,
        last_error TEXT
      );
    ''');
  }
}
```

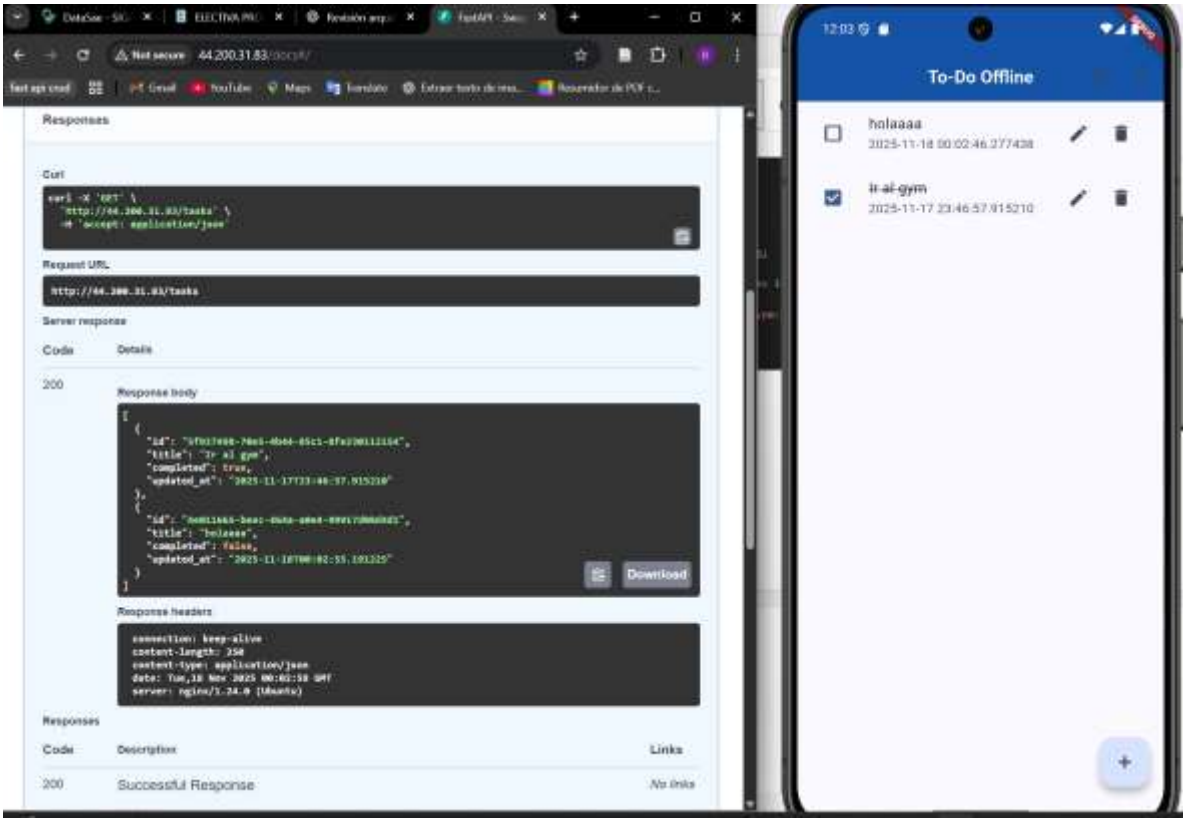
Flujo de sincronización

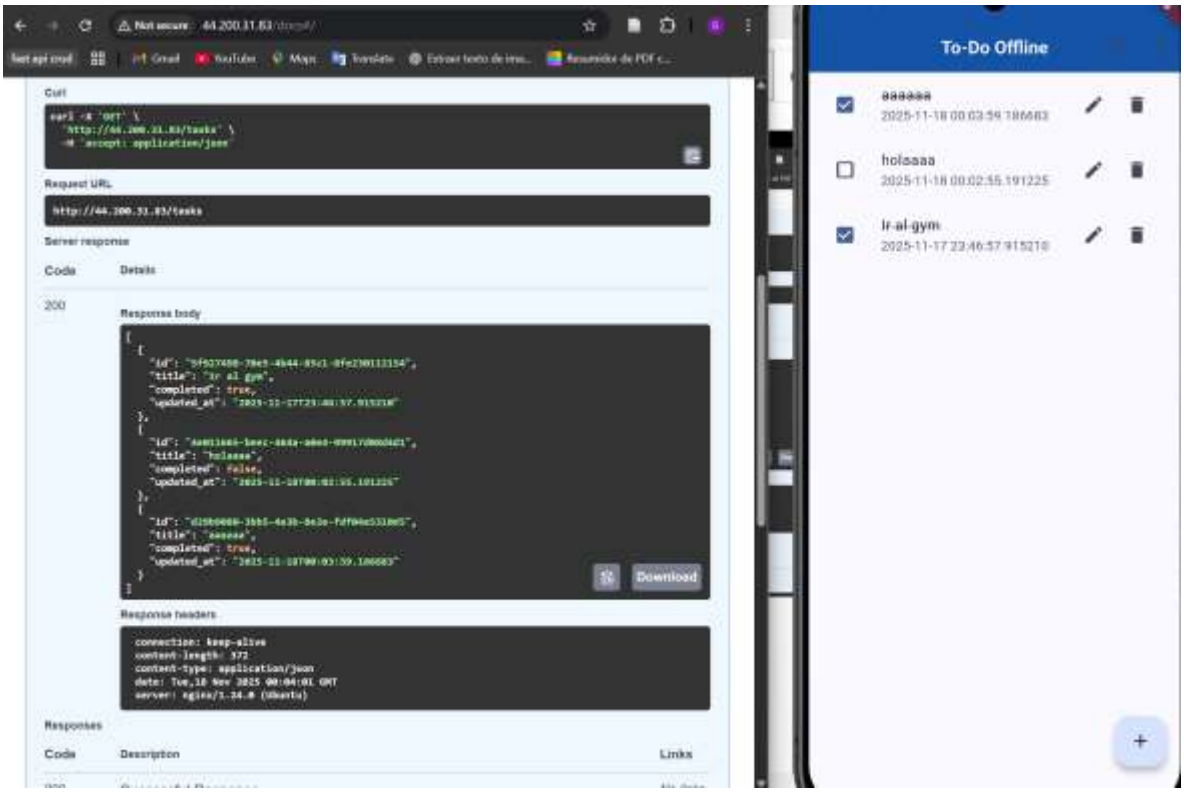
1. La app detecta conectividad (connectivity_plus) o al abrirse.
2. Toma operaciones pendientes y las envía al backend.
3. Si la API responde con éxito, marca la operación como completada y actualiza syncedAt.
4. Si falla, reintenta con backoff exponencial y registra last_error.

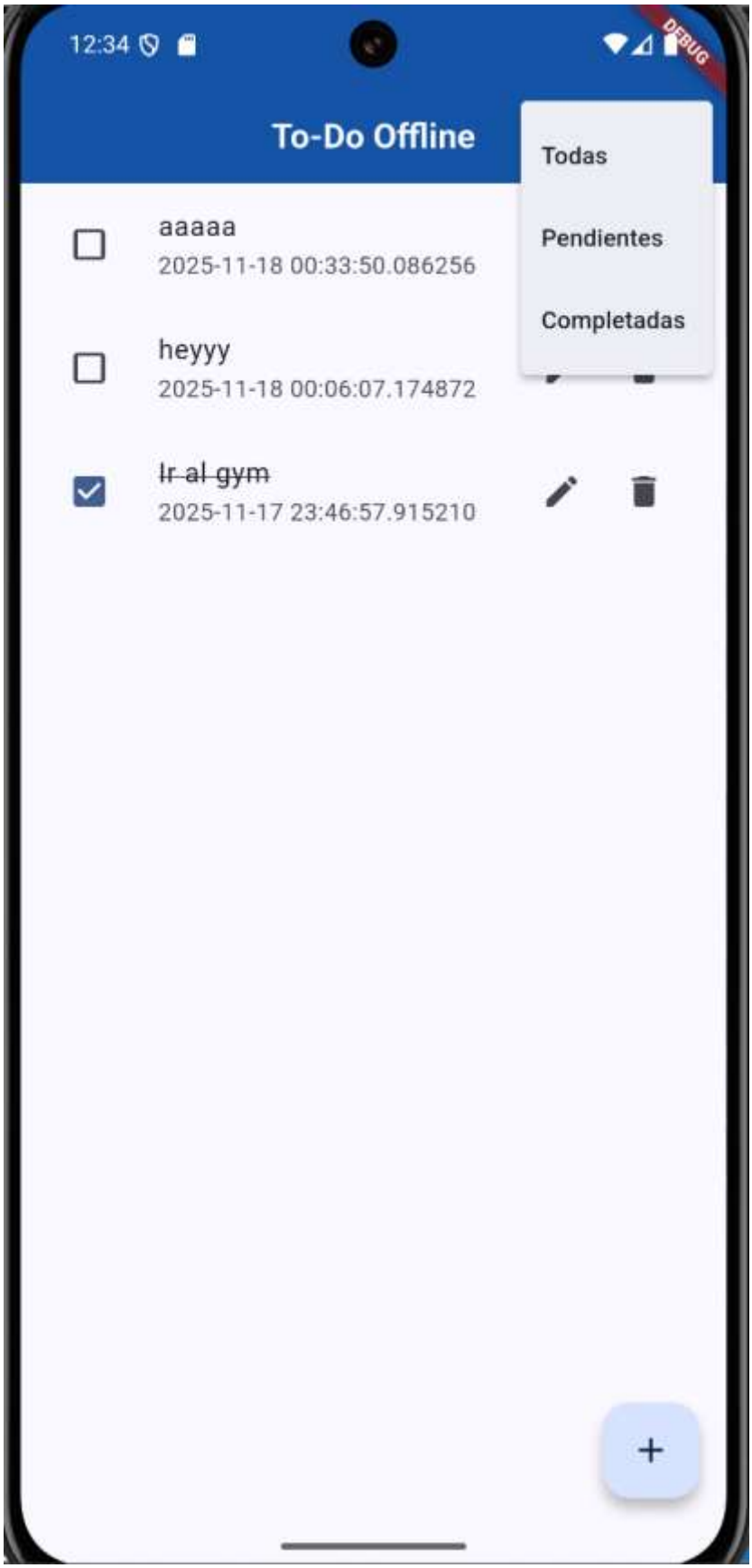


```dart
class SyncService {
  // ------------------------------------------------------------
  // AUTO-SYNC (cada X segundos si hay internet)
  // ------------------------------------------------------------
  void startAutoSync({int intervalSeconds = 8}) {
    _timer?.cancel();
    _timer = Timer.periodic(Duration(seconds: intervalSeconds), (_) async {
      final conn = await Connectivity().checkConnectivity();
      if (conn != ConnectivityResult.none) {    The type of the right operand ('ConnectivityResult'
        await syncQueue();
      }
    }); // Timer.periodic
  }

  void stop() {
    _timer?.cancel();
    _running = false;
  }

  // ------------------------------------------------------------
  // PROCESAR COLA
  // ------------------------------------------------------------
  Future<void> syncQueue() async {
    if (_running) return;
    _running = true;

    final queue = await local.getQueue();

    for (final op in queue) {
      final opId = op['id'] as String;
      final opType = op['op'] as String;
      final entityId = op['entity_id'] as String;
      final payload = op['payload'] as String;
      int attempt = (op['attempt_count'] ?? 0) as int;

      _logger.i("Processing queue item: $opId type:$opType attempt:$attempt");

      TaskModel? model;

```

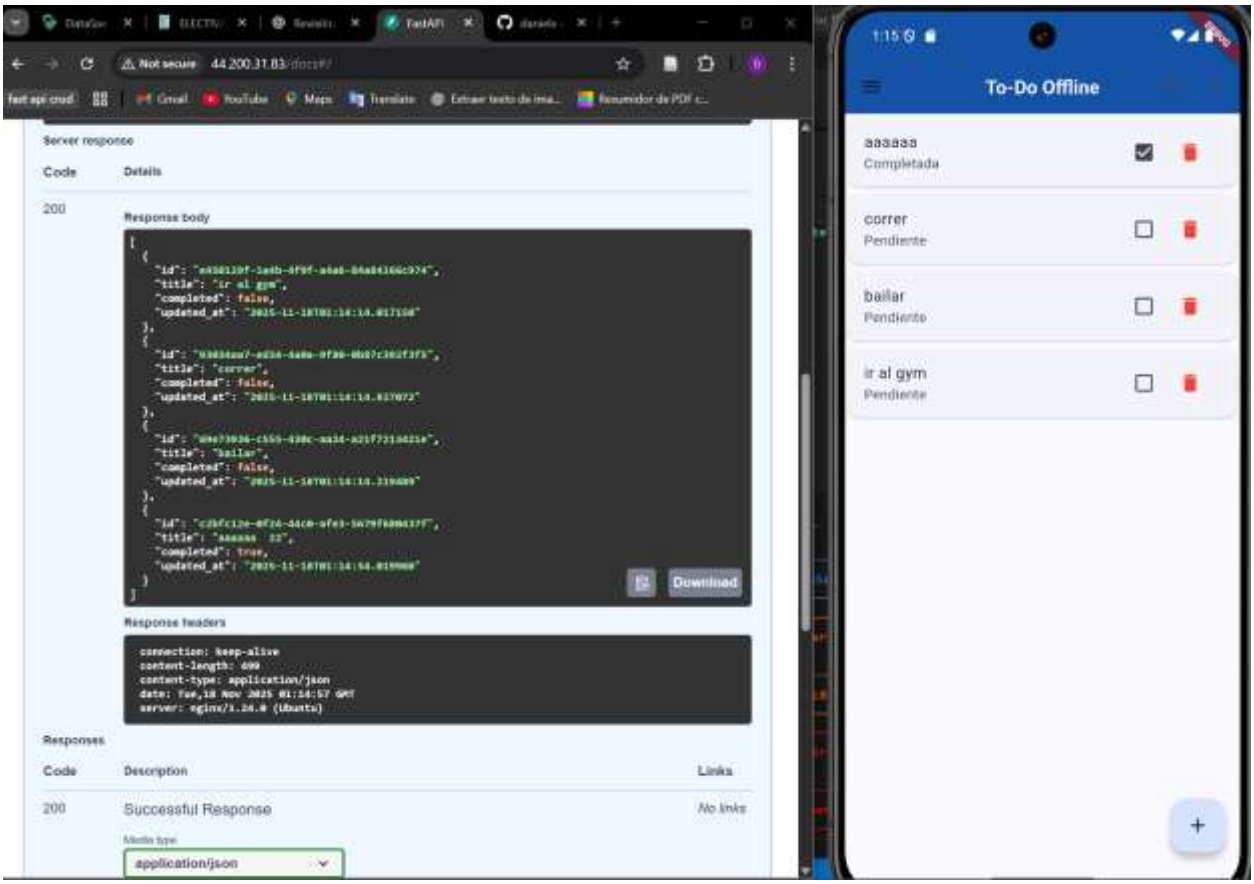
Comunicación con la API:





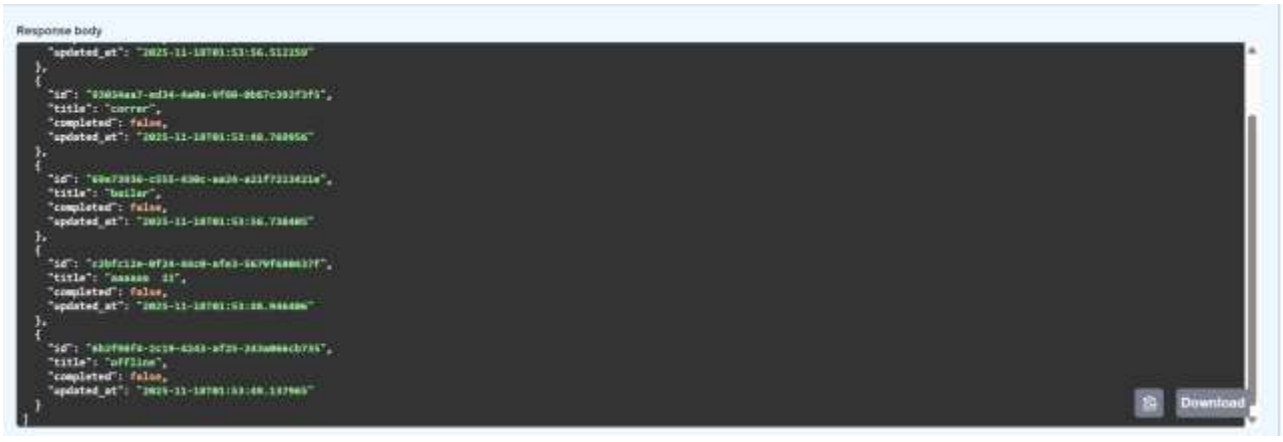

12:34
DEBUG
To-Do Offline
Todas
Pendientes
Completadas
aaaaa
2025-11-18 00:33:50.086256
heyyy
2025-11-18 00:06:07.174872
~~Ir al gym~~
2025-11-17 23:46:57.915210

UI final:

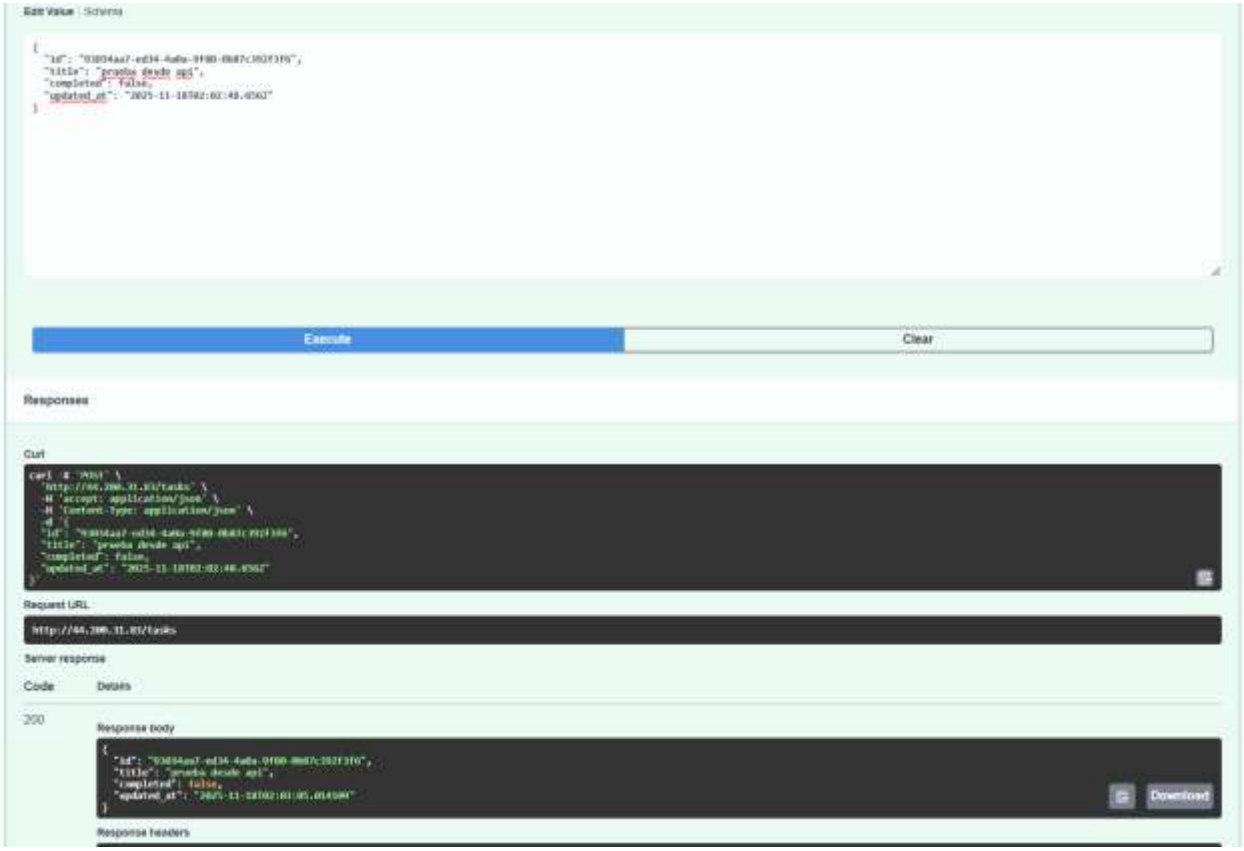


Se hacen las pruebas respectivas desde un celular Android y funciona offline, una vez regresa el internet se sincroniza:

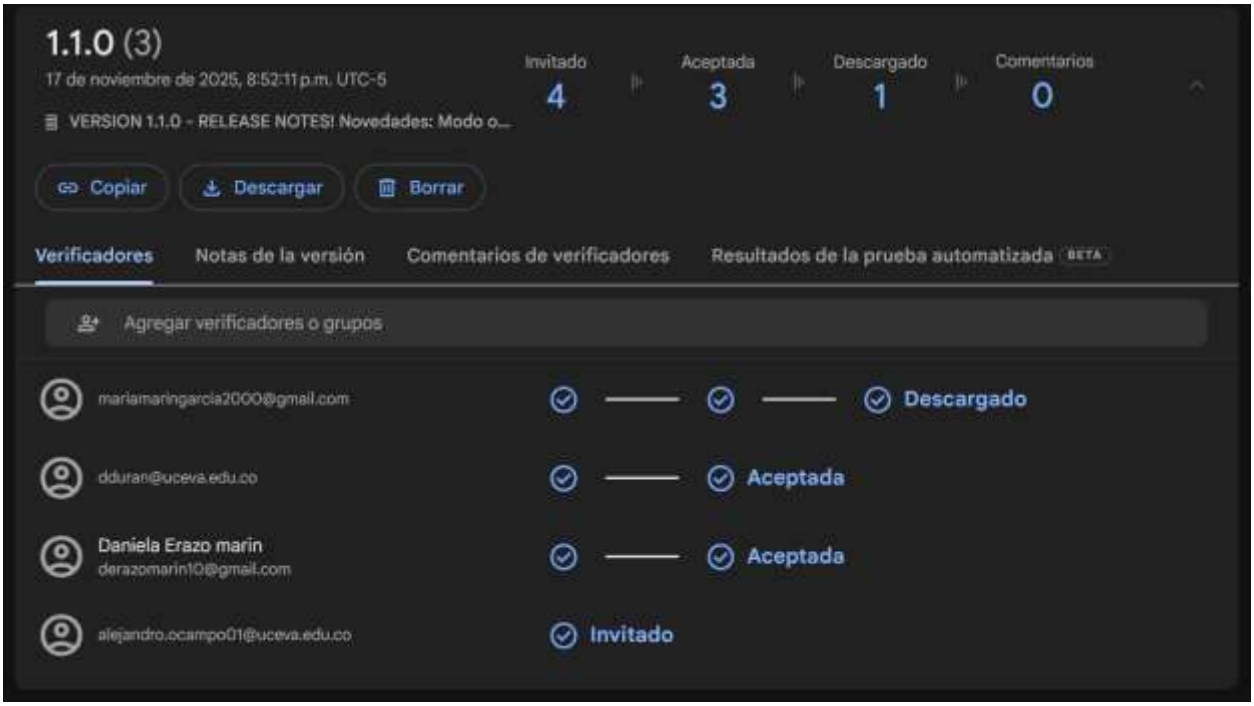
Api: http://44.200.31.83/docs#/default/get_all_tasks_tasks_get

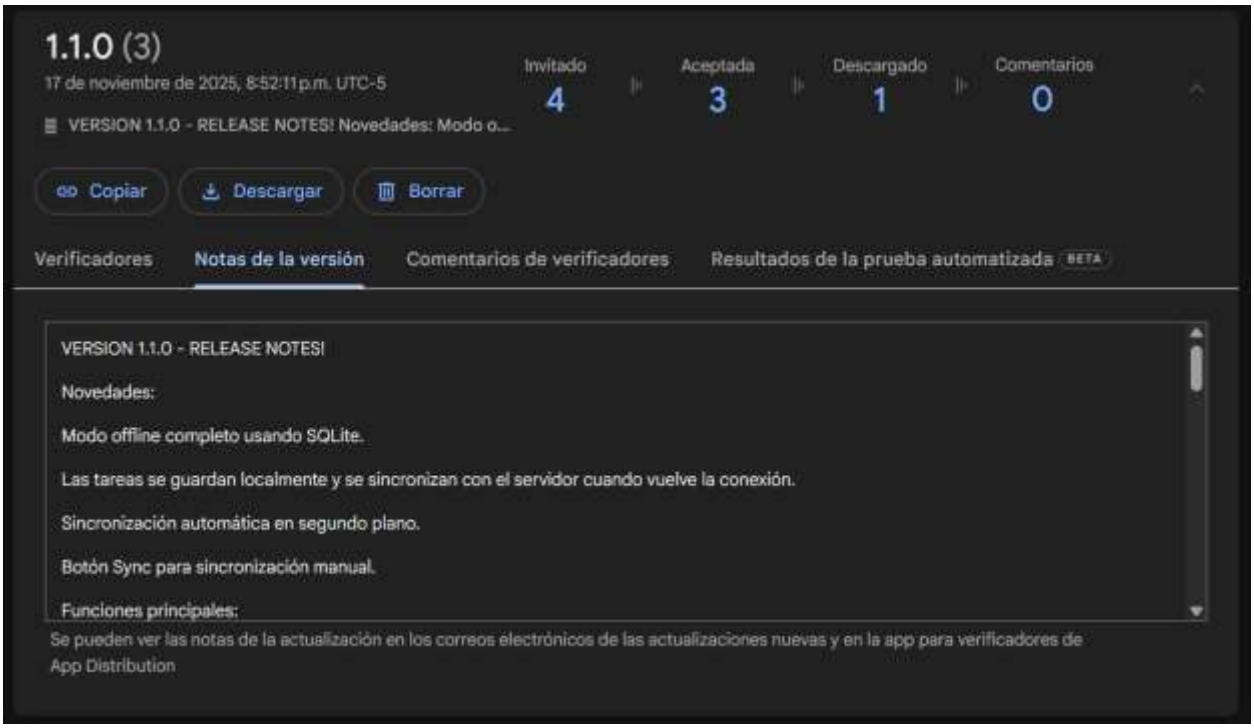


Prueba desde API:



6 Generación de APK





VERSION 1.1.0 - RELEASE NOTES!

Novedades:

- Modo offline completo usando SQLite.
- Las tareas se guardan localmente y se sincronizan con el servidor cuando vuelve la conexión.
- Sincronización automática en segundo plano.
- Botón Sync para sincronización manual.

Funciones principales:

- Crear tareas.
- Editar tareas.
- Marcar tareas como completadas o pendientes.
- Eliminar tareas.
- Filtrar por todas, pendientes o completadas.
- Lista actualizada en tiempo real.

Arquitectura:

- Manejo de estado con Provider.
- Repositorio con datasources local (SQLite) y remoto (API AWS).
- Servicio de sincronización de operaciones en cola.

Mejoras incluidas:

- Corrección de duplicados al sincronizar.
- Mejor rendimiento al cargar y refrescar tareas.
- La UI ahora se actualiza correctamente después de eliminar o cambiar el estado de una tarea.
- Manejo optimizado del campo updatedAt.

Cómo probar:

- Para probar el modo offline:
- Desactiva la conexión.
- Crea o edita tareas
- Reactiva internet.
- Pulsa el botón Sync y verifica que los cambios llegan al servidor.

En modo online:

- Las tareas se sincronizan automáticamente y SQLite se mantiene actualizado.

Errores conocidos:

- En algunos dispositivos el check de completado puede tardar un momento en reflejarse mientras se recarga la lista desde SQLite.
- La sincronización puede ser lenta con conexiones inestables.

Próximas mejoras:

- Modo oscuro.
- Notificaciones.
- Subtareas y prioridades.
- Sistema de usuarios.